

CSCI 5540 – Fall 26

Tutorial 4

DENG Yangtao

Credits to CMU Machine Learning Systems

What we will cover

1

Data parallelism

Different data, one synchronized model

2

Gradient communication

AllReduce, ReduceScatter, AllGather

3

Training memory

Parameters are only part of the budget

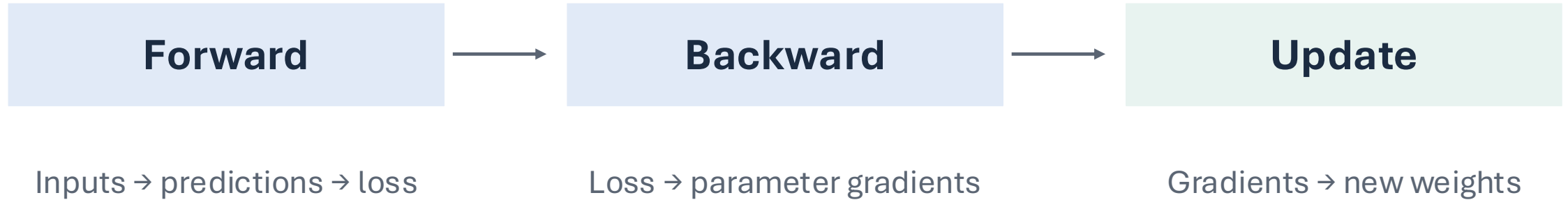
4

ZeRO stages 1–3

Partition redundant model state

Main question: what must be replicated, and what can be shared?

A training step has three stages



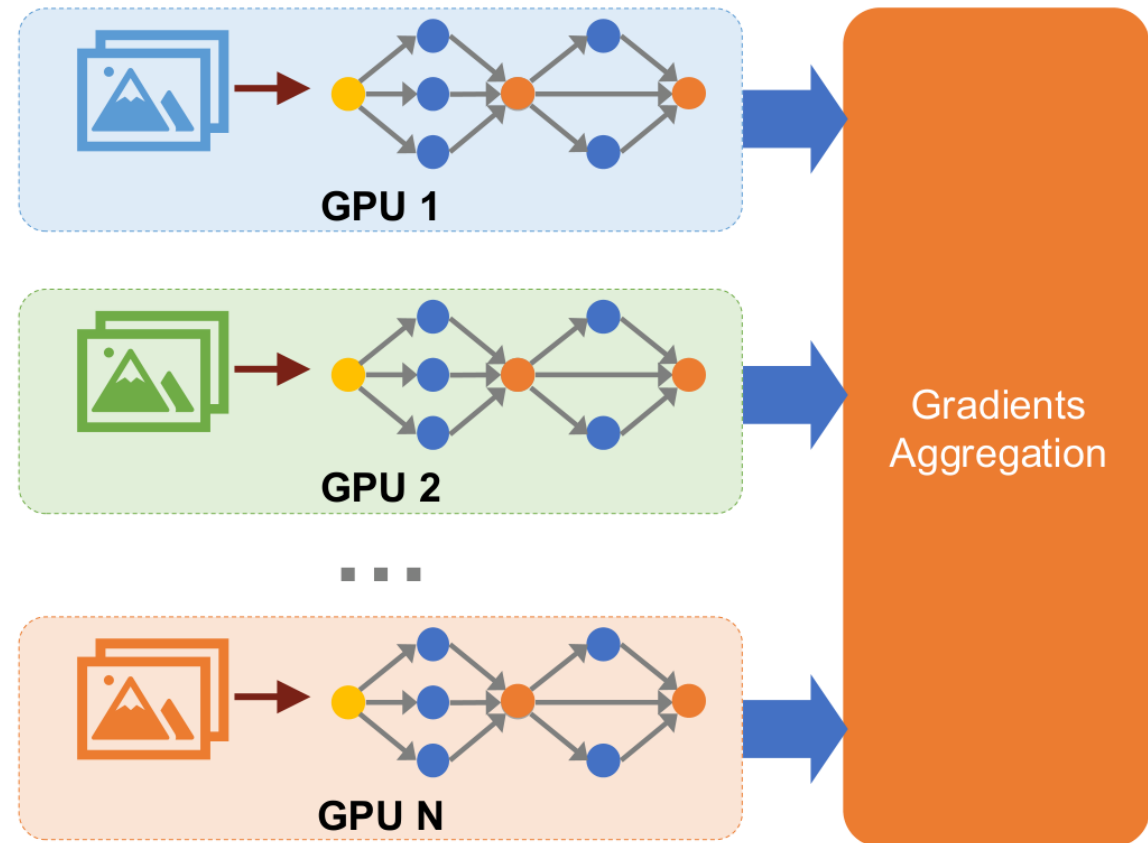
$$g = \frac{1}{B} \sum_{j=1}^B \nabla_w \ell_j(w), \quad w_{\text{new}} = w - \eta g$$

Independent examples create parallel work; the update needs a shared gradient.

Data parallelism: same model, different data

On every GPU

- Same initial parameters
- Different local mini-batch
- Full forward + backward
- Same averaged gradient
- Same optimizer update



Global batch size = number of workers × local batch size

Why do we average gradients?

GPU 0

Local mean gradient: 2

GPU 1

Local mean gradient: 6



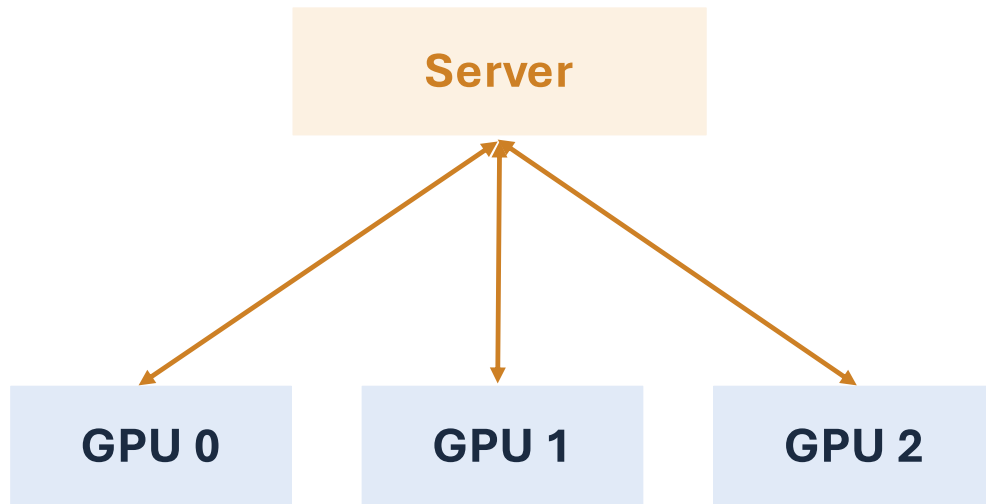
Global mean: $(2 + 6) / 2 = 4$

Both GPUs: $w = 10 - 0.1 \times 4 = 9.6$

Equal local batch sizes → average local means. Unequal sizes → weight by sample count.

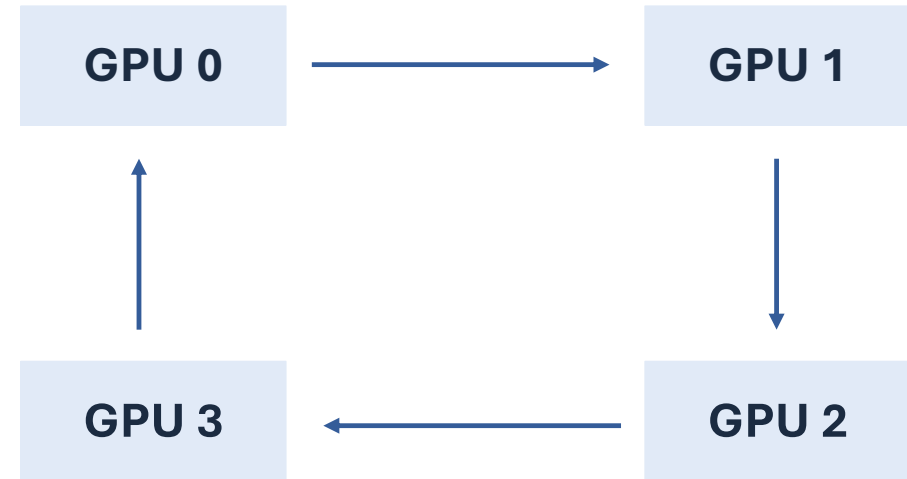
Who combines the gradients?

Parameter server



Central endpoint can limit throughput

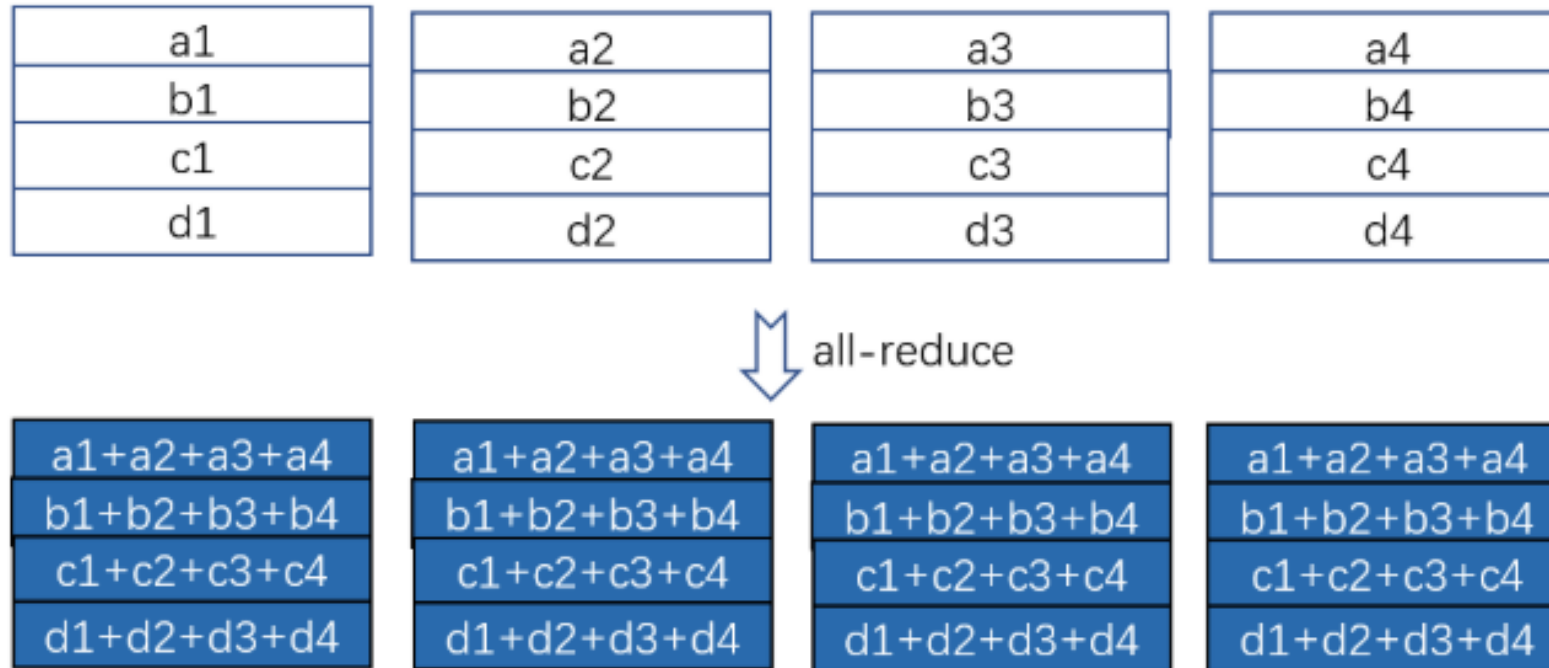
Worker collective



Workers share communication work

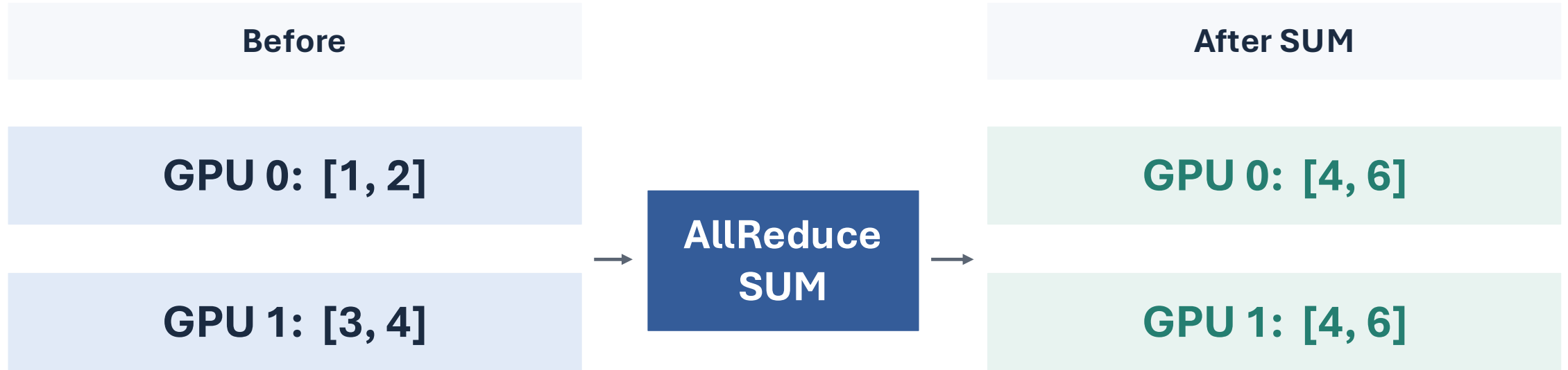
AllReduce produces the same reduced vector on every participating GPU.

Who combines the gradients?



AllReduce produces the same reduced vector on every participating GPU.

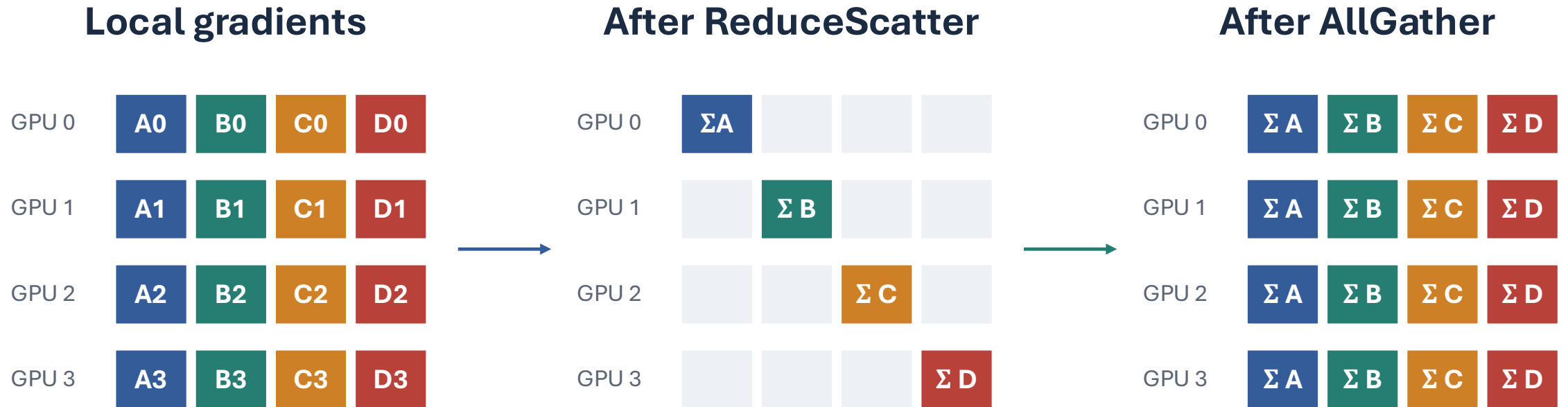
AllReduce is a contract, not one algorithm



Average gradient on both GPUs: [2, 3]

The output semantics stay the same when the collective algorithm (Ring, tree, and butterfly) changes.

Ring AllReduce = ReduceScatter + AllGather

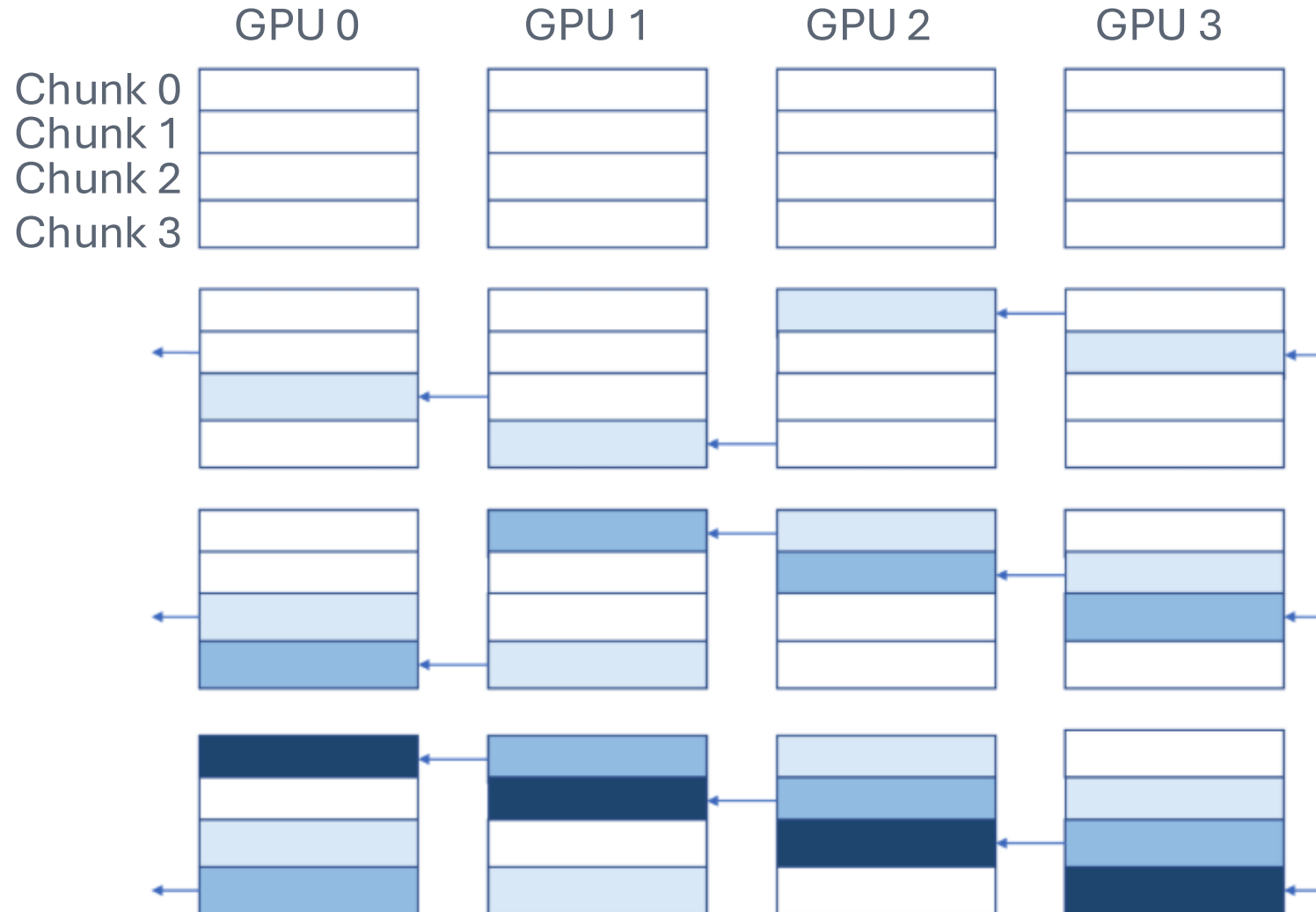


Each phase: $N - 1$ neighbor-exchange rounds

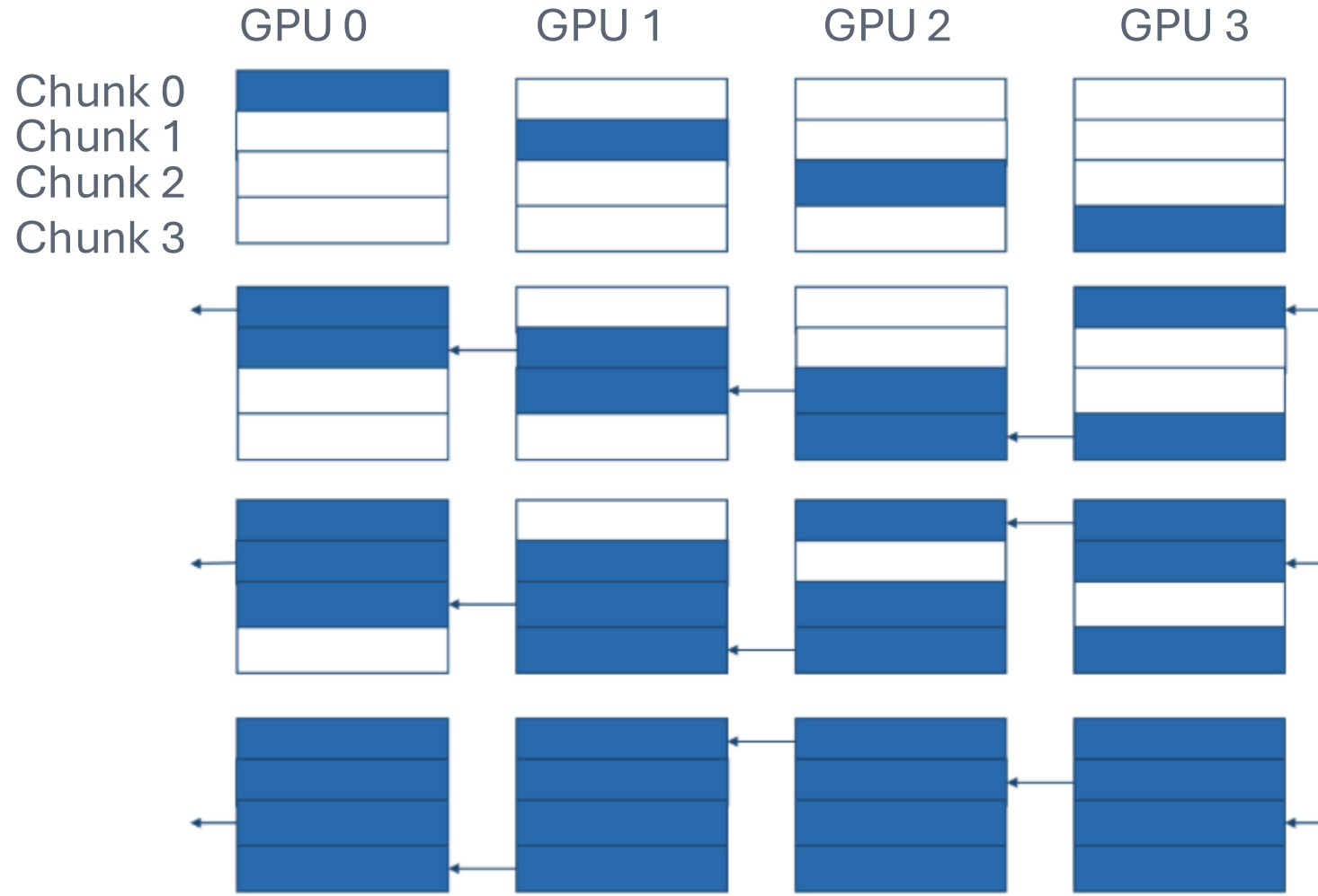
Σ = sum of that chunk across all GPUs; divide by N for the mean

ReduceScatter reduces and partitions. AllGather replicates the reduced chunks.

Ring Reduce-Scatter example: 4 chunks around 4 GPUs



Ring All-Gather example: 4 chunks around 4 GPUs

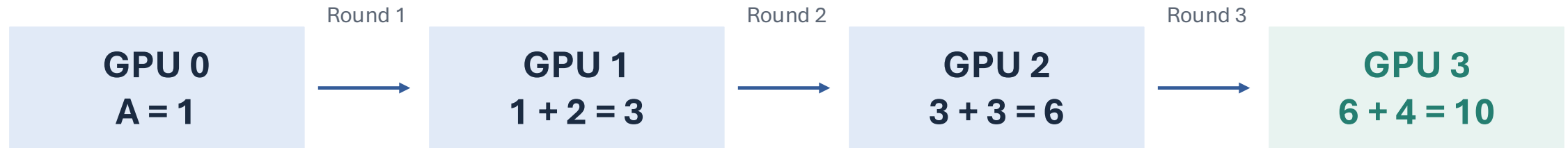


Ring example: follow one chunk around 4 GPUs

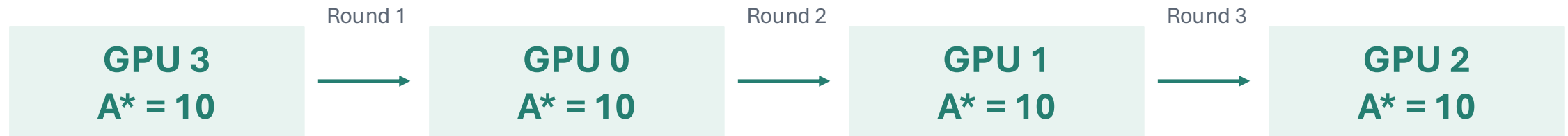
Chunk A contributions: GPU 0 = 1, GPU 1 = 2, GPU 2 = 3, GPU 3 = 4

Ring direction: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$

1. ReduceScatter: send and add



2. AllGather: copy the complete sum



3 + 3 rounds. Sum = 10 everywhere; mean gradient = $10 / 4 = 2.5$.

Why ring communication scales well

$$V_{\text{send/GPU}} = 2(N - 1) \frac{S}{N} \longrightarrow 2S$$

4 GPUs

1 GB gradient vector

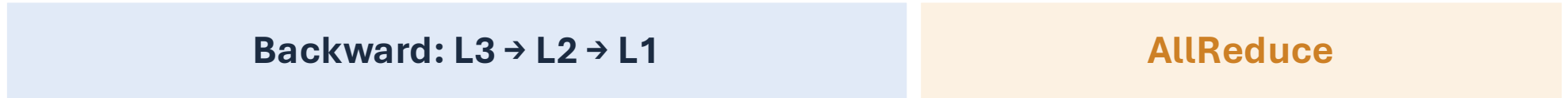
1.5 GB sent / GPU

- Balanced bandwidth demand; no central server bottleneck
- More GPUs still mean more message rounds
- Tensor size and network topology affect the best algorithm

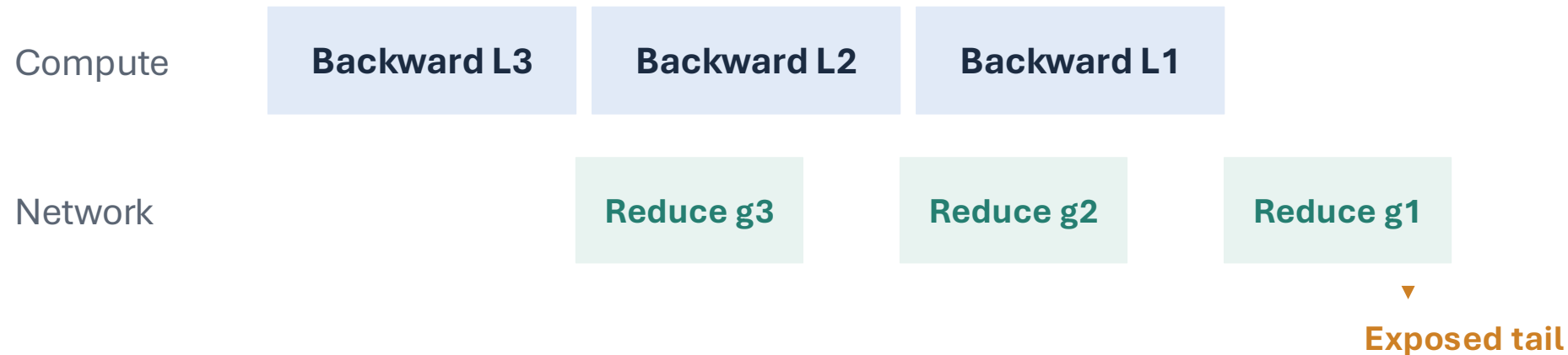
Bandwidth efficiency is different from “fastest in every setting.”

Communication can overlap with backward

Wait until backward finishes

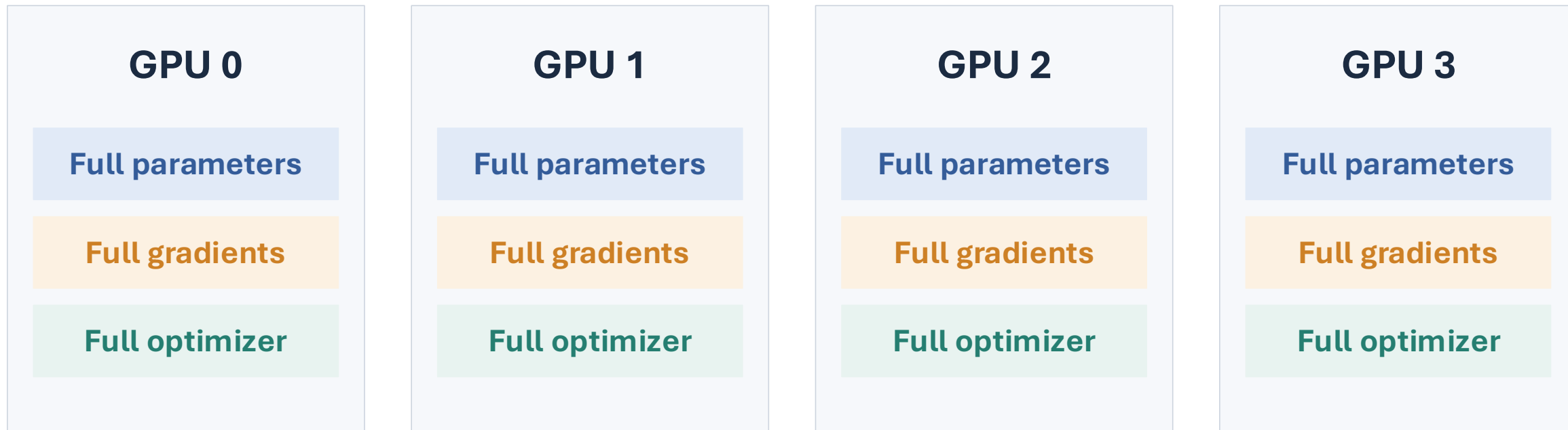


Overlap ready gradient buckets



Only exposed communication extends step time; overlap depends on the workload.

More GPUs do not automatically fix OOM



Four replicas $\neq$ one model spread across four memories

ZeRO targets redundant model state across data-parallel workers.

ZeRO removes redundancy in three stages

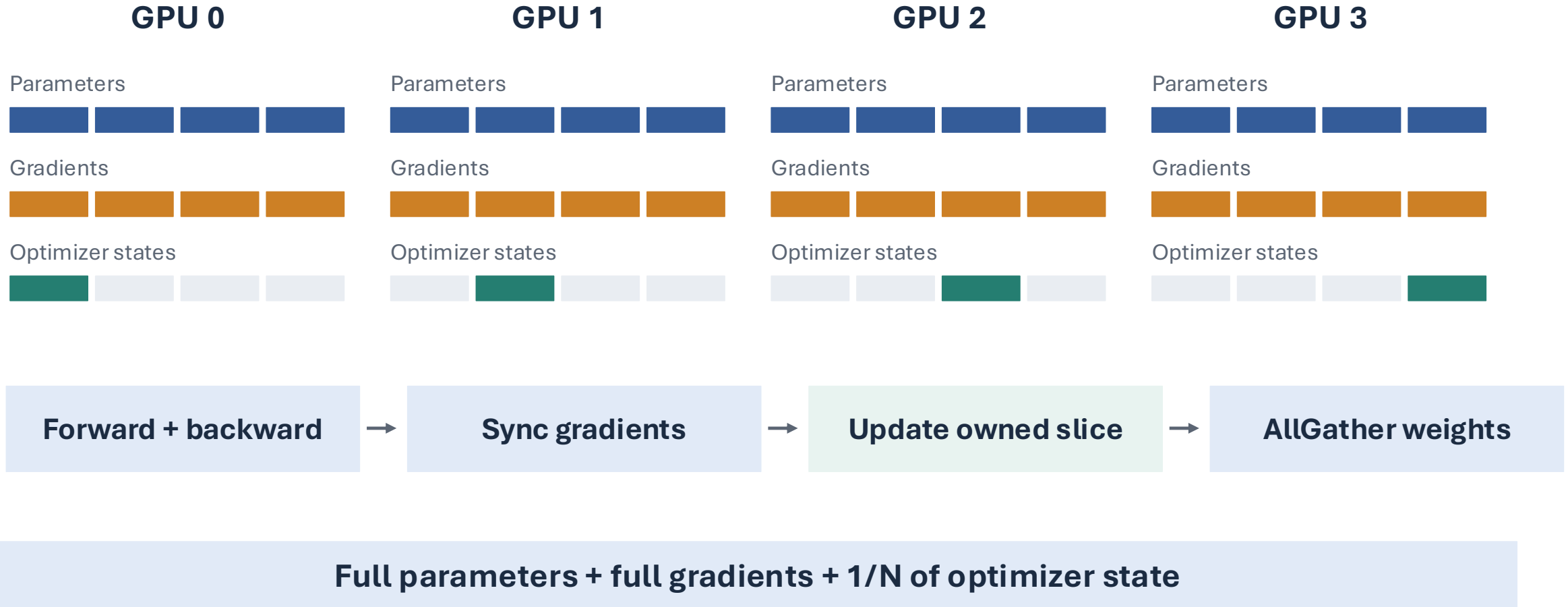
State	DP	ZeRO-1	ZeRO-2	ZeRO-3
Parameters	Full	Full	Full	Shard
Gradients	Full	Full	Shard	Shard
Optimizer state	Full	Shard	Shard	Shard

“Shard” = one distinct slice per GPU; together, all slices form the complete state.

The stages change storage ownership, not which training examples a GPU processes.

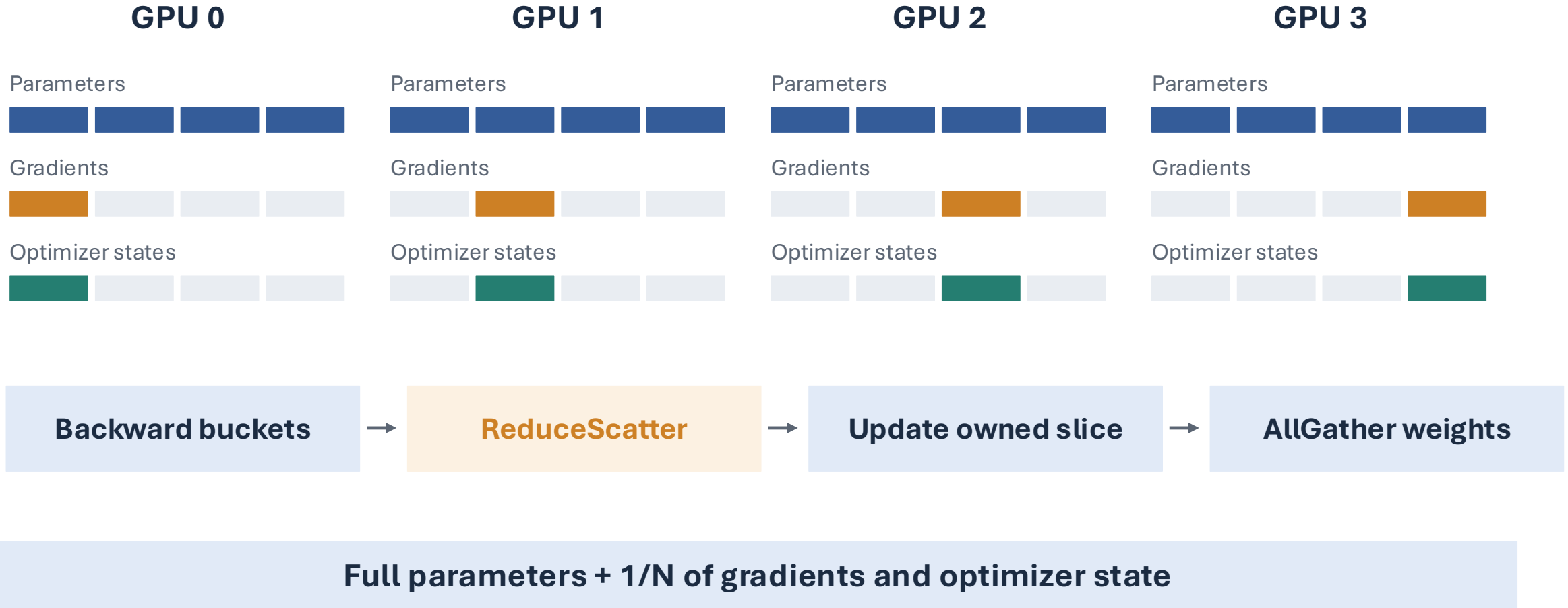
ZeRO-1: partition the optimizer state

Four GPUs; each colored cell is one quarter of the model state.



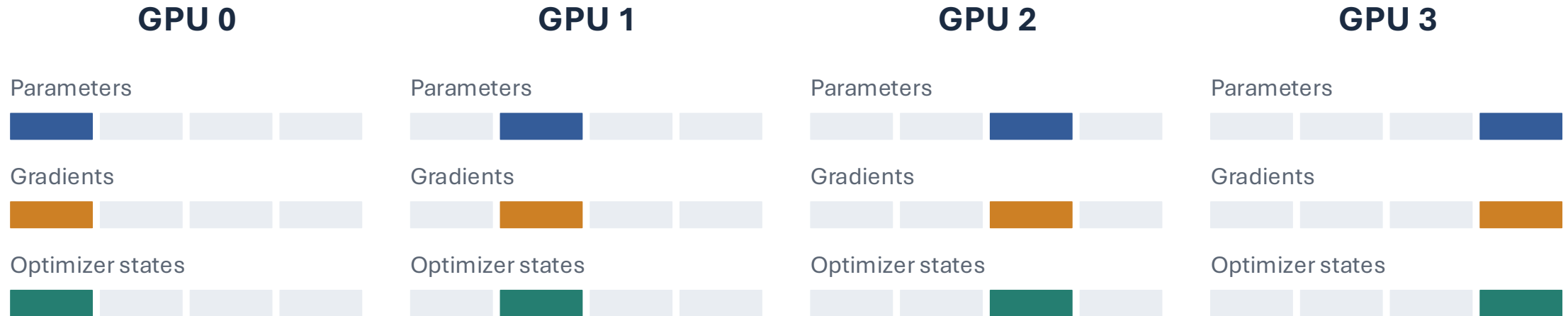
ZeRO-2: keep only the gradients you own

Four GPUs; each colored cell is one quarter of the model state.



ZeRO-3: gather parameters just in time

Four GPUs; each colored cell is one quarter of the model state.



AllGather layer weights



Compute this layer



Release full weights

Repeat during backward if weights were released; ReduceScatter the gradients.

1/N of all persistent model state — plus temporary gathered weights and activations

ZeRO-3 example: 2 GPUs, 2 layers

GPU 0 owns W1

GPU 1 owns W2

Both GPUs compute L1 and L2 on their own data. Owners keep their weights.

Step	Need weights	Gather transfer	Release after compute
Forward L1	W1	GPU 0 → GPU 1	GPU 1 drops W1
Forward L2	W2	GPU 1 → GPU 0	GPU 0 drops W2
Backward L2	W2 again	GPU 1 → GPU 0	GPU 0 drops W2
Backward L1	W1 again	GPU 0 → GPU 1	GPU 1 drops W1

Backward: reduce dW2 to GPU 1 and dW1 to GPU 0; each owner updates its weights.

Saving persistent memory adds transfers and temporary working copies.

Worked example: 1B parameters, 4 GPUs

M = parameter count; N = GPUs; formulas in bytes; 1 GB = 10^9 bytes

Method	Model-state bytes / GPU	At M= 10^9 , N=4	
DP	16M		16.0 GB
ZeRO-1	4M + 12M/N		7.0 GB
ZeRO-2	2M + 14M/N		5.5 GB
ZeRO-3	16M/N		4.0 GB

16 GB → 7 GB → 5.5 GB → 4 GB per GPU

Model state only: actual peak memory is higher and implementation-dependent.

Which stage should we use?

Full training state fits

DP may be sufficient

Optimizer / gradient state dominates

Try ZeRO-1 or ZeRO-2

Replicated parameters do not fit

Consider ZeRO-3

Also check activations, batch size, temporary buffers, and network bandwidth.

Choose using both memory measurements and end-to-end step time.

Quick check: explain the ownership

- 1 After ReduceScatter, what does each GPU hold?
- 2 In ZeRO-2, which states are sharded — and which are full?
- 3 Why does ZeRO-3 still need temporary parameter memory?

Use the words “replica”, “owner”, and “temporary copy” in your answers.

Three ideas to remember

1

Data parallelism

Split examples; synchronize gradients and updates.

2

Collective decomposition

AllReduce = ReduceScatter + AllGather.

3

ZeRO

Shard optimizer state → gradients → parameters.

Thanks!